KERTENKELE VERİSİYLE KARAR AĞACI OLUŞTURMA

Bilgisayar Destekli Matematik Öğretimi Final Ödevi

Araç: Google Colab (Python)
Veri Kümesi: lizard_data.csv (320 gözlem, 7 değişken)

Bu Ödev Ne Anlam İfade Ediyor?

Bu dönem boyunca üç farklı ortamda karar ağacı oluşturdunuz:

- Data kartlarıyla elle karar ağacı (kağıt üzerinde)
- CODAP ortamında sürükle-bırak karar ağacı (bilgisayarda görsel)
- Google Colab'de Python ile karar ağacı (kod yazarak)

Final ödevinizde bu üç deneyimi birleştiriyorsunuz. Yeni bir veri kümesi olan kertenkele verisini kullanarak dersimizde yaptığımız adımların aynısını gerçekleştirecek ve modelinizin performansını yorumlamalısınız.

Bu dokümandaki kodları kopyala-yapıştır yaparken girinti (boşluk) kayması nedeniyle hata alabilirsiniz. Çalışan ve doğrulanmış kodlara ders boyunca birlikte oluşturduğumuz ve sizinle paylaşılan Google Colab not defterinden ulaşabilirsiniz.

ADIM 1: VERİYİ TANIMA

Veri kümesiyle tanışmadan model kurmak, soruyu okumadan sınava girmek gibidir.

1.1 Kertenkele Verisi Nedir?

Bu veri kümesinde üç farklı kertenkele türüne ait 320 bireyin fiziksel ölçümleri yer almaktadır.

Değişken Adı	Türü	Değer Aralığı / Kategoriler	Açıklama
Tur	Kategorik	Anolis / Sceloporus / Uta	Kertenkele türü
Cinsiyet	Kategorik	Dişi / Erkek	Biyolojik cinsiyet
GovdeUzunlugu	Sayısal (sürekli)	36.4 – 83.3 mm	Gövde boyu
ToplamUzunluk	Sayısal (sürekli)	76.4 – 209.4 mm	Kuyruk dahil toplam boy
Kutle	Sayısal (sürekli)	2.57 – 25.93 g	Ağırlık
BacakUzunlugu	Sayısal (sürekli)	13.8 – 43.9 mm	Arka bacak uzunluğu
YasamAlani	Kategorik	Yer / Kaya / Ağaç	Kertenkelenin yaşadığı yer

Veriyi inceledikten sonra şunu düşünün:
Bir kertenkelenin ölçümlerine bakarak onun hakkında ne öğrenebiliriz? Hangi soruları yanıtlayabiliriz?

1.2 Veri Kümesini Colab'a Yükleme

Her yeni Colab oturumunda veri setini tekrar yüklemeniz gerekir!

1. Sol kenardaki 📁 (Dosyalar) simgesine tıklayın.
2. Açılan panelin üzerine veri setini sürükleyip bırakın.
3. Yükleme tamamlandıktan sonra aşağıdaki kodları çalıştırın.

1.3 Veriyi Yükleme ve İlk Bakış

Dersimizde yiyecek verisiyle yaptığımızın aynısını şimdi kertenkele verisiyle yapıyoruz:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
```

```
df = pd.read_csv("lizard-data.csv")
df.head()          # İlk 5 satır
```

Ardından şu soruları koda dökerek cevaplayın:

```
print('Satır sayısı:', df.shape[0])
print('Sütun sayısı:', df.shape[1])
print(df.columns.tolist())
df['YasamAlani'].value_counts()   # Her sınıftan kaç tane var?
df.describe().round(1)            # Sayısal özet
```

Yanıtlamanız gereken sorular (Her bir sorunun cevabını Colab'e (not defterinize) yazın):

- Veri kümesinde kaç kertenkele var?
- Hangi yaşam alanı en kalabalık? Hangisi en az?
- Bu dengesizlik modelimizi nasıl etkileyebilir? (Dersimizdeki "sınıf dengesi" tartışmasını hatırlayın.)
- Kertenkelelerin ortalama ağırlığı nedir? Bu değer üç tür için farklı mıdır? Nedenini açıklayın.

Oluşturmanız Gereken Karar Ağaçları

Bu ödevde aynı veri setiyle farklı sorular soracak ve her biri için ayrı bir karar ağacı kuracaksınız. Hangi soruyu sorduğunuza göre hedef değişkeniniz değişecek ama kullandığınız veri ve kod yapısı aynı kalacak.

- Veriyi inceledikten sonra şunu düşünün: **Bu kertenkeleler hakkında ne tahmin etmek isterdiniz?**

Kendi sorularınızı oluşturun ve her biri için bir karar ağacı kurun:

- Veri setine bakarak **en az üç soru** yazın. Her soru bir hedef değişkene karşılık gelmelidir.
- Her soru için ayrı bir karar ağacı kurun.

İpucu: İyi bir soru, veri setindeki bir sütunu hedef alır ve "Bu kertenkelenin ... ne olabilir?" şeklinde kurulur. Hangi sütunların hedef olabileceğini veri tablosuna bakarak kendiniz keşfedebilirsiniz.

ADIM 2: VERİYİ MODELİN ANLAYACAĞI HALİNE GETİRME

Karar ağacı sayılarla çalışır, metin sütunları ve hedef değişken ayrı tutulmalıdır.

2.1 Özellikler (X) ve Hedef (y) Ayrımı

Dersimizde yiyecek verisiyle şöyle yapmıştık: *"Öneri"* sütunu hedef (y), geriye kalan sütunlar özellikler (X) oluyordu. Kertenkele verisinde de aynı mantığı uyguluyoruz — tek fark, bu sefer hangi soruyu sorduğumuza göre hedef değişken değişiyor.

Dikkat!

Karar ağacı yalnızca sayılarla çalışır. *Tur* ve *Cinsiyet* gibi metin sütunları doğrudan modele verilemez. Önce 0/1 değerlerine dönüştürülmesi gerekir. Bunu her ağaç kodunda `pd.get_dummies()` ile otomatik olarak yapıyoruz:

```
# pd.get_dummies() metin sütunlarını otomatik olarak 0/1 sütunlarına çevirir.
# Örneğin: Tur → Tur_Anolis, Tur_Sceloporus, Tur_Uta
X = pd.get_dummies(df[ozellikler])
```

Her ağaç kodu bu satırı kendi içinde çalıştırıyor; ayrı bir dönüştürme adımına gerek yok.

Neden get_dummies kullandık?

Karar ağacı “Anolis > Uta” gibi bir karşılaştırma yapamaz, sayılar istemez. `pd.get_dummies()` her kategoriyi ayrı bir 0/1 sütununa çevirir. Örneğin “Tur” sütunu → Tur_Anolis, Tur_Sceloporus, Tur_Uta sütunlarına dönüşür.

Her ağaç kodunun başında hedef değişkenini değiştirerek X ve y otomatik olarak yeniden tanımlanır, ayrı bir adıma gerek yoktur.

ADIM 3: EĞİTİM / TEST AYRIMI

Modeli sınavda görmediği sorularla sınamak — CODAP dersinden tanıdık bir adım (Choosy).

3.1 Veriyi %80 Eğitim / %20 Test Olarak Bölme

Bu adımı dersimizde şöyle öğrenmiştik: Modele hem ders notu veririz (eğitim), hem de sınava sokarız (test). Sınav sorularını önceden görmüş olursa başarısı gerçeği yansıtmaz. Bu yüzden veri setini eğitim ve test olarak ayırmalıyız.

Aşağıdaki örnek *yaşam alanı* tahmini için yapılmıştır. **Diğer ağaçlarda aynı adımları farklı hedef değişkenlerle tekrarlamalısınız**.

X ve y değişkenleri her ağaç kodunun başında tanımlanmaktadır. Bu nedenle aşağıdaki kodu çalıştırmadan önce ADIM 4'teki ağaç kodlarından birini en az bir kez çalıştırmanız gerekir.

```
X_egitim, X_test, y_egitim, y_test = train_test_split(
    X, y,
    test_size=0.2,       # %20 test olarak böl
    random_state=42,     # Tekrarlanabilirlik için sabit sayı
    stratify=y           # Her sınıftan orantılı böl
)
```

```
print('Eğitim seti:', len(X_egitim), 'kertenkele')
print('Test seti:  ', len(X_test),   'kertenkele')
print()
print('Eğitim seti sınıf dağılımı:')
print(y_egitim.value_counts())
```

Veri setini bölerken yanıtlamanız gereken sorular:

- `stratify=y` parametresini kaldırırsanız ne değişebilir? Deneyin ve farkı açıklayın.
- `random_state=42` yerine `random_state=7` yazarsanız sonuçlar değişir mi? Deneyin ve nedenini açıklayın.

ADIM 4: KARAR AĞACI MODELI KURMA VE DEĞERLENDİRME

Farklı derinlikleri deneyin — en iyi derinliği veriye sordurarak bulun.

4.1 Farklı Derinlikleri Test Etme

Dersimizde yiyecek verisiyle max_depth parametresini 1'den 10'a kadar deneyip her derinlikteki doğruluğu karşılaştırmıştık. Şimdi aynısını kertenkele verisiyle yapıyorsunuz:

```
derinlikler = [1, 2, 3, 4, 5]  # Derinliği for döngüsü için siz belirleyin [1, 2, 3, ..., 10]
print(f"{'Derinlik':>10} {'Eğitim MCR':>12} {'Test MCR':>10}")           # sütun başlıkları
```

```
for derinlik in derinlikler:
    agac = DecisionTreeClassifier(max_depth=derinlik, random_state=42)
    agac.fit(X_egitim, y_egitim)
    egitim_mcr = 1 - accuracy_score(y_egitim, agac.predict(X_egitim))
    test_mcr = 1 - accuracy_score(y_test, agac.predict(X_test))
    print(f"{derinlik:>10} {egitim_mcr:>12.3f} {test_mcr:>10.3f}")
```

MCR değerlerini inceleyerek yanıtlamanız gereken sorular:

- Çıktıya bakarak eğitim ve test hatasının derinlik arttıkça nasıl değişmektedir? Eğitim ve test değerlerine bakarak gerekçenizi yazın.
- Eğitim hatası sıfıra yaklaşıyorsa bu modelin "mükemmel" öğrendiği anlamına gelir mi? Sizce ne anlama geliyor?
- Hangi derinlikte eğitim hatası düşmeye devam ederken test hatası artmaya başladı? O satırda modele ne olduğunu açıklayın.
- Yalnızca bu sayılara bakarak hangi derinliği seçerdiniz? Hem eğitim hem test hatasını göz önünde bulundurarak gerekçenizi yazın.
- En düşük test hatası (MCR) ile en düşük eğitim hatası farklı derinliklerde mi çıktı? Bu size ne söylüyor?

4.2 Farklı Derinlikleri Grafikte Gösterme

```
egitim_mcr = []
test_mcr = []
```

```
for derinlik in derinlikler:
    agac = DecisionTreeClassifier(max_depth=derinlik, random_state=42)
    agac.fit(X_egitim,y_egitim)
    egitim_mcr.append(1 - accuracy_score(y_egitim, agac.predict(X_egitim)))
    test_mcr.append(  1 - accuracy_score(y_test,agac.predict(X_test)))
```

```
plt.figure(figsize=(8, 5))
plt.plot(derinlikler, egitim_mcr, marker='o', label='Eğitim MCR')
plt.plot(derinlikler, test_mcr, marker='o', label='Test MCR')
plt.xlabel('Ağacın Derinliği (max_depth)')
plt.ylabel('Yanlış Sınıflandırma Oranı (MCR)')
plt.title('Derinliğe Göre Model Hata Oranı')
plt.legend()
plt.grid(True)
plt.show()
```

Grafiği inceleyerek yanıtlamanız gereken sorular:

- Grafikteki iki eğri birbirinden ne zaman ayrılmaya başlıyor? Bu ayrılma noktası size ne söylüyor?
- Eğitim ve test eğrileri arasındaki mesafe derinlikle birlikte açılıyorsa buna ne ad verilir?
- Grafiğe bakarak hangi max_depth değerini seçerdiniz? Sayısal tablodaki kararınızla aynı mı?
- Kurduğunuz her karar ağacı için bu grafiği ayrı ayrı çizin. Optimum derinlik her hedef için aynı mı çıktı? Farklılaştıysa neden?

ADIM 5: OPTİMUM KARAR AĞACINI OLUŞTURMA

Ağacınızı çizin ve CODAP'ta kurduğunuzla karşılaştırın.

5.1 Optimum Derinlikte Final Ağacı Kurma

Grafikteki bulgularınıza dayanarak optimum derinliği seçin ve final modelini kurun:

```
optimum_derinlik = ...  # Grafiğe bakarak siz belirleyin
final_agac =DecisionTreeClassifier(max_depth=optimum_derinlik, random_state=42)
final_agac.fit(X_egitim, y_egitim)
```

```
# Test verisinin MCR'si
y_tahmin = final_agac.predict(X_test)
test_mcr = 1 - accuracy_score(y_test, y_tahmin)
print(f'Optimum Derinlik: {optimum_derinlik}')
print(f'Test Hatası (MCR): {test_mcr:.4f}'
```

5.2 Ağacı Çizme

```
plt.figure(figsize=(10, 5))
plot_tree(
    final_agac,
    feature_names=X.columns.tolist(),
    class_names=list(final_agac.classes_.astype(str)),      # sınıf isimlerini metne çevirir
    filled=True,
    rounded=True,
    fontsize=9
)
plt.title(f'Karar Ağacı — max_depth={optimum_derinlik}')
plt.tight_layout()
plt.show()
```

Ağacı yorumlama soruları:

- Kök düğümde hangi değişken kullanılmış? Neden tam da bu değişkenin ilk sırada yer aldığını düşünüyorsunuz?
- Ağaç yapraktan köke doğru okunduğunda bir kertenkeleyi sınıflandırmak için kaç karar yeterli oluyor? Bu sayı size ne söylüyor?
- Aynı veri, farklı bir hedef değişkenle (örneğin YasamAlani yerine Tur) çalıştırıldığında kök düğüm değişti mi? Değiştiyse bunu nasıl yorumlarsınız?
- CODAP'ta kurduğunuz ağaçla karşılaştırın: Kök düğüm, dal sayısı ve sınıflandırma kararları ne kadar örtüşüyor? Farklılık varsa nereden kaynaklanıyor olabilir?

ADIM 6: YAZILI DEĞERLENDİRME VE YANSIMA

Bulgularınızı cümlelerle aktarın, her bir soruyu açıklayın.

Aşağıdaki soruların her birine kendi cümlelerinizle yanıt verin. Yanıtlarınızı koda yorum satırı olarak (#) veya Colab'da Metin (Markdown) hücresi olarak ekleyin.

Soru 1 — Öğrenme Yolculuğu

Bu dönemde data kartları, CODAP ve Python olmak üç farklı ortamda karar ağacı oluşturdunuz. Aşağıdaki soruları bu ortamları göz önünde bulundurarak yanıtlayın:

- Her ortam size farklı bir şey kattı mı? Her bir ortamın neler kattığını yazın.
- Hangi ortamda *"modelim ne yapıyor"* sorusunu daha iyi anladınız? Nedeninizi açıklayın.
- Hangi ortam size daha kolay geldi? Bu kolaylık gerçekten anladığınızı mı, yoksa sadece alıştığınızı mı gösteriyor? Nedeninizi açıklayın.
- Hangi ortamda en çok zorlandınız ve bu zorluk neden kaynaklanıyordu? Aracın kendisi mi, yoksa karar ağacı kavramının o ortamda daha mı görünmez olması mıydı? Nedeninizi açıklayın.
- Yaptığınız bir hatayı hangi ortamda daha kolay fark ettiniz? Sizi uyaran şey aracın kendisi miydi, yoksa başka bir şey mi?
- Bir ortamda rahatlıkla yapabildiğiniz ama bir diğerinde hiç yapamadığınız bir şey oldu mu?
- Üç ortamdan birinin diğerinin yerini tutamayacağı bir durum var mı? Nedeninizi açıklayın.

Soru 2 — Veri Anlayışı

Kertenkele verisindeki üç yaşam alanı (Yer / Kaya / Ağaç) ile fiziksel özellikler arasındaki ilişkiyi düşünün:

- Modelinizin kök düğümde seçtiği değişken sizi şaşırttı mı? Biyolojik açıdan bu seçim mantıklı mı?
- Ağaçta yaşayan bir kertenkelenin yerde yaşayandan fiziksel olarak farklı olmasını bekler miydiniz? Modeliniz bu farkı yakalayabildi mi?
- Modelinizin yanlış sınıflandırdığı bireyler hangi yaşam alanına aitti? Bunun biyolojik bir açıklaması olabilir mi?
- Veriyi ilk gördüğünüzde kendiniz hangi değişkenin en belirleyici olacağını tahmin etmiştiniz? Modelinizin bulgusuyla örtüştü mü?

Soru 3 — Model Performansı

Seçtiğiniz optimum derinlikte elde ettiğiniz MCR değerini yazın ve gerekçelendirerek yanıtlayın:

- Hangi sınıf en iyi, hangisi en kötü tahmin edildi? Bunun bir açıklaması var mı?
- MCR değeriniz sizi tatmin etti mi? "İyi" bir model için bu değerin ne olması gerektiğini düşünüyorsunuz?
- Aynı veriyle farklı bir hedef değişken seçseydiniz (Tur ya da Cinsiyet) MCR değeri değişir miydi? Değiştiyse neden?
- Modelin başarısız olduğu durumlar size veri hakkında ne söylüyor?

Soru 4 — Karar Ağaçlarının Sınırları

Bu ödevde karar ağacının her zaman mükemmel çalışmadığını gördünüz:

- Modelinizin hata yaptığı durumlar rastgele mi görünüyor, yoksa belirli bir örüntü var mı?
- Bu modelin daha iyi sonuç verebilmesi için neler yapılabilir? Yalnızca teknik değil, verinin kendisiyle ilgili ne değişebilir?
- Karar ağacı yerine farklı bir model kullansaydınız sonuç değişir miydi? Neden?
- Bu deneyimden sonra bir arkadaşınıza *"karar ağacı nedir, ne işe yarar, nerede yetersiz kalır?"* diye sorsalar ne söylerdiniz?

Teslim Kontrol Listesi

Ödevi tamamlamadan önce aşağıdaki tüm maddeleri tamamladığınızdan emin olun:

☐	Adım 1: Veriyi inceleyerek yükledim, `shape` ve `value_counts` çıktılarını aldım, tüm soruları yanıtladım.
☐	Adım 2: Kategorik değişkenleri one-hot encoding ile dönüştürdüm, X ve y'yi oluşturdum, tüm soruları yanıtladım.
☐	Adım 3: Veriyi %80/%20 olarak böldüm, sınıf dağılımlarını kontrol ettim, tüm soruları yanıtladım.
☐	Adım 4: `max_depth` 1'den 10'a kadar döngüyle denedim, eğitim/test doğruluk grafiğini çizdim, tüm soruları yanıtladım.
☐	Adım 4: Grafiğe bakarak optimum derinliği seçtim ve tüm soruları yanıtladım.
☐	Adım 5: Final ağacımı `plot_tree` ile görselleştirdim ve kök değişkeni yorumladım, tüm soruları yanıtladım.
☐	Adım 6: Yazılı değerlendirme sorularının hepsini gerekçelendirerek kendi cümlelerimle açıkladım.
☐	Tüm kodları çalıştırdım, hiç hata almadım (ya da hataları düzelttim).
☐	Colab not defterimi `.ipynb` formatında indirdim, dosyayı `ad-soyad.ipynb` olarak yeniden adlandırdım ve e-posta ile gönderdim.

Değerlendirme Rubriği

Kriter	Yetersiz (0-2)	Gelişiyor (3-4)	Yetkin (5-6)	Mükemmel (7-8)
Veri Keşfi (Adım 1-2)	Kod çalışmıyor ya da eksik	Kodlar çalışıyor ama yorum yok	Çıktılar doğru, kısa yorum var	Eksiksiz keşif + anlamlı yorum
Model Kurma (Adım 3-4)	Bölme ya da döngü eksik	Tek derinlik denenmiş	Döngü var, grafik doğru	Optimum seçim gerekçeli
Görselleştirme (Adım 5)	Ağaç çizilmemiş	Çizilmiş ama yorumsuz	Ağaç + kısa yorum	CODAP karşılaştırması var
Yazılı Yanıtlar (Adım 6)	1-2 soru yanıtlanmış	3 soru yüzeysel	4 soru yanıtlanmış	Derin, özgün ve eleştirel yanıtlar

Kodda takıldığınız her yerde hata mesajını okuyun. Hata mesajında takıldığınız yerin cevabı saklıdır. Derste de dediğimiz gibi hata mesajları problemi tarif eden dostunuzdur, düşmanınız değil.